

本地大模型 RAG 增强完整落地教程

部分内容由豆包生成

 **适用技术栈：**Ollama 本地大模型 + Chroma 向量库 + SQLite 结构化数据库 + Java 后端

教程目标：从 0 到 1 搭建生产级双数据源增强 RAG 系统，包含查询增强、多路召回、Rerank 重排序、动态 SQL 生成、双数据源融合全部模块

一、整体架构与核心概念

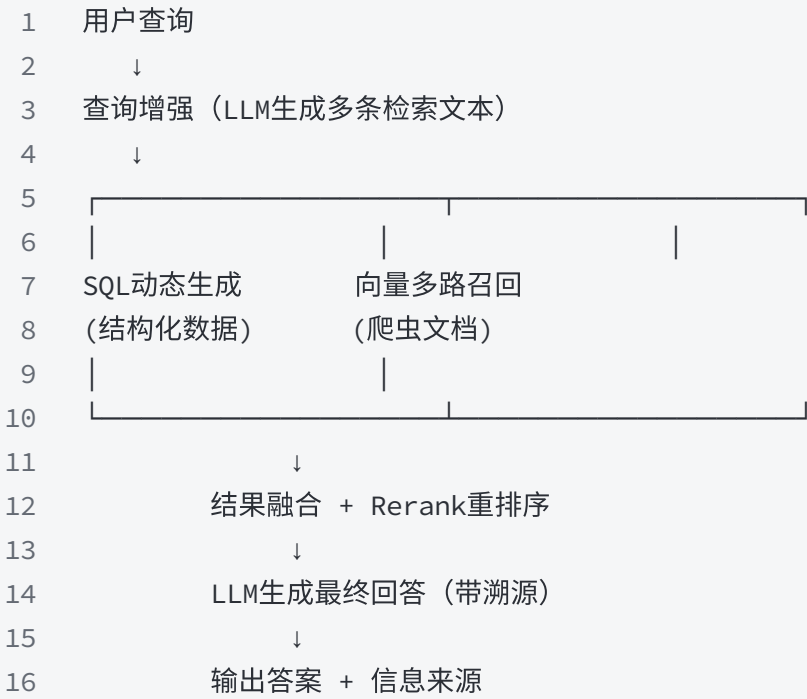
1.1 什么是增强 RAG

普通简易 RAG 的流程是：用户提问 → 关键词向量检索 → 直接丢给 LLM 生成回答。这种方式存在召回率低、无关文档干扰、结构化数据缺失等问题。

增强 RAG（生产级） = 查询增强 + 多路检索增强 + 文档切片/重排增强 + 生成约束增强，四大模块协同工作，大幅提升回答准确率和相关性。

1.2 完整架构图

RAG 增强架构流程



1.3 四大增强模块对比

增强模块	解决的问题	效果提升
查询增强	用户口语化提问、关键词不准导致漏匹配	召回率提升 30%~50%
多路检索	单条查询召回不全，结构化数据查不到	信息完整度提升 60%+
Rerank 重排序	向量语义匹配但实际无关的垃圾文档	准确率提升 20%~40%
生成约束	LLM 编造内容、幻觉、答非所问	幻觉率降低 70%+

二、环境准备

2.1 软件依赖

- **JDK 17+**：Java 开发环境
- **Maven 3.8+**：项目构建工具
- **Ollama**：本地大模型运行时
- **ChromaDB**：向量数据库

2.2 模型准备

拉取所需模型

```
1  # 生成回答用的大模型 (推荐 qwen3 或 deepseek-r1)
2  ollama pull qwen3:latest
3
4  # 向量嵌入模型 (中文优化推荐 bge-m3)
5  ollama pull bge-m3:latest
```

2.3 启动 Chroma 向量数据库

启动 Chroma 服务

```
1  # 安装 chroma (Python 环境)
2  pip install chromadb
3
4  # 启动服务, 指定数据存储路径
5  chroma run --path ./chroma_data --host 0.0.0.0 --port 8000
```

三、模块一：查询增强（Query Enhancement）

3.1 核心原理

使用 LLM 对用户原始查询进行优化扩展，生成多条不同角度的检索文本，分别向量化后检索向量库，合并去重，大幅提升召回率。

3.2 三种增强策略

1. **精简核心关键词**：去除语气词、修饰词，保留最核心的检索语义
2. **补充专业术语**：扩展同义词、相关技术名词，扩大召回范围
3. **换角度提问**：拆分子问题视角，覆盖不同的表达方式

3.3 Java 代码实现

QueryEnhancer.java 查询增强器

```
1  package com.rag.enhance;
2
3  import com.alibaba.fastjson2.JSON;
4  import com.rag.llm.OllamaClient;
5  import java.util.List;
6
7  public class QueryEnhancer {
8      private final OllamaClient ollama = new OllamaClient();
9
10     /**
11      * 对用户查询进行增强，生成多条检索文本
12      * @param userQuery 用户原始查询
13      * @return 增强后的查询列表
14      */
15     public List<String> enhance(String userQuery) throws Exception {
16         String prompt = ""
17             用户问题：%s
18             任务：生成3条不同角度检索文本用于向量库匹配，仅输出JSON字符串数组，无多余文字。
19             1.第一条：精简核心关键词
20             2.第二条：补充专业场景术语
21             3.第三条：拆分子问题视角
22
23             示例输入：ag-psd本地vue项目怎么引入
24             示例输出：["ag-psd本地项目接入教程","vite vue集成ag-psd完整代码","ag-psd安装踩坑与配置"]
25         """.formatted(userQuery);
26
27         String res = ollama.chat(prompt);
```

```
28         int start = res.indexOf("[");
29         int end = res.lastIndexOf("]") + 1;
30         return JSON.parseArray(res.substring(start,
        end)).toJavaList(String.class);
31     }
32 }
```

四、模块二：向量多路召回（Vector Multi-Recall）

4.1 文档预处理

爬虫抓取的网页/文档原始文本噪音极大（广告、换行、多余标签、长文本），必须预处理切片再入库。

预处理流程

原始文档 → 清洗（去换行/空格/无效标签） → 分段切片（Chunk） → 重叠窗口（上下文不割裂） → 入库向量库

切片参数建议

文档类型	Chunk 大小	重叠大小
技术文档/教程	400~500 字符	80~100 字符
代码示例文档	500~600 字符	100~150 字符
短问答/FAQ	200~300 字符	40~60 字符

4.2 多路召回实现

将查询增强生成的多条检索文本，分别向量化检索向量库，合并所有结果，去重后得到多路召回的文档集合。

多路向量检索代码

```
1 // 多路召回
2 List<String> allRawDocs = new ArrayList<>();
3 for (String query : enhancedQueries) {
4     List<String> docs = vectorStore.search(query, TOP_K_PER_QUERY);
5     allRawDocs.addAll(docs);
6 }
7
8 // 去重
```

```

9 List<String> uniqueDocs = new ArrayList<>();
10 for (String doc : allRawDocs) {
11     if (!uniqueDocs.contains(doc)) {
12         uniqueDocs.add(doc);
13     }
14 }

```

五、模块三：Rerank 文档重排序

5.1 为什么需要重排序

单纯向量相似度会出现语义匹配但实际无关的文本，比如：用户问"ag-psd 怎么安装"，可能检索到"psd-tools Python 安装教程"，因为语义上都是"安装 PSD 工具"，但实际答非所问。

Rerank 的作用：使用 LLM 对所有召回文档与用户原问题打分，只保留 Top N 高相关文档，过滤噪音，提升最终回答的准确率。

5.2 Java 代码实现

Reranker.java 文档重排序器

```

1 package com.rag.enhance;
2
3 import com.alibaba.fastjson2.JSON;
4 import com.rag.llm.OllamaClient;
5 import java.util.List;
6
7 public class Reranker {
8     private final OllamaClient ollama = new OllamaClient();
9
10    /**
11     * 对检索结果进行重排序，保留相关性最高的 topN 个文档
12     * @param userQuery 用户原始查询
13     * @param allDocs 待排序的文档列表
14     * @param topN 保留数量
15     * @return 排序后的文档列表
16     */
17    public List<String> rerank(String userQuery, List<String> allDocs, int
topN) throws Exception {
18        StringBuilder docSb = new StringBuilder();
19        for (int i = 0; i < allDocs.size(); i++) {
20            docSb.append(i + 1).append(".
21            ").append(allDocs.get(i)).append("\n===\n");
22        }

```

```

23         String prompt = """
24     用户原始问题: %s
25     候选文档列表:
26     %s
27     任务: 给每个文档打分0-10分, 分数越高相关性越强。
28     只返回相关性最高前%d个文档原文, JSON数组输出, 无多余文字。
29     """.formatted(userQuery, docSb, topN);
30
31         String jsonStr = ollama.chat(prompt);
32         int start = jsonStr.indexOf("[");
33         int end = jsonStr.lastIndexOf("]") + 1;
34         return JSON.parseArray(jsonStr.substring(start,
35             end)).toJavaList(String.class);
36     }

```

 **优化建议:** 生产环境推荐使用 bge-reranker 等专用重排序模型, 速度更快、效果更稳定。
LLM rerank 适合快速原型和小数据量场景。

六、模块四：动态 SQL 生成（结构化数据检索）

6.1 核心思路

将数据库表结构、字段说明、业务规则提供给 LLM, 让 LLM 根据用户问题自动生成安全的 SELECT SQL 语句, 查询结构化业务数据。

6.2 安全机制（必须实现）

1. **只读限制:** 只允许 SELECT, 禁止 DROP/DELETE/UPDATE/ALTER 等修改语句
2. **参数化查询:** 所有条件使用 ? 占位符, 禁止字符串拼接, 防 SQL 注入
3. **强制 LIMIT:** 所有查询必须带 LIMIT, 防止全表扫描拖垮数据库
4. **黑名单校验:** 程序层拦截危险关键字, 双重防护
5. **只读账号:** 数据库连接账号只授予 SELECT 权限

6.3 Java 代码实现

SqlGenerator.java SQL动态生成器

```

1  package com.rag.sql;
2
3  import com.alibaba.fastjson2.JSON;

```

```

4  import com.alibaba.fastjson2.JSONObject;
5  import com.rag.config.RagConfig;
6  import com.rag.llm.OllamaClient;
7
8  public class SqlGenerator {
9      private final OllamaClient ollama = new OllamaClient();
10
11      public SqlResp generate(String userQuery) throws Exception {
12          String prompt = ""
13  # 角色
14  你是专业SQL生成助手，仅输出可执行SELECT SQL，禁止解释、多余文字，只返回json字符串。
15
16  # 数据库Schema
17  %s
18
19  # 强制规则
20  1. 仅生成SELECT语句，不允许任何修改、删除、建表语句；
21  2. 所有查询末尾必须加 LIMIT 10；
22  3. 查询条件全部使用 ? 占位符，禁止直接拼接用户输入；
23  4. 字段、表名严格按照上面schema，不存在的字段不能写；
24  5. 模糊查询使用 LIKE ?，参数值是 %%关键词%%；
25  6. 输出固定JSON格式，key只有两个：sql、params
26
27  # 用户查询问题
28  %s
29
30  示例输出: {"sql":"select name,lang from sdk_info where lang = ? limit
31  10","params":["TS"]}
32
33      """".formatted(RagConfig.DB_SCHEMA, userQuery);
34
35      String jsonStr = ollama.chat(prompt);
36      int start = jsonStr.indexOf("{");
37      int end = jsonStr.lastIndexOf("}") + 1;
38      JSONObject json = JSON.parseObject(jsonStr.substring(start, end));
39
40      SqlResp resp = new SqlResp();
41      resp.sql = json.getString("sql");
42      resp.params = json.getJSONArray("params").toArray(new String[0]);
43      return resp;
44  }
45
46  // SQL安全校验
47  public boolean securityCheck(String sql) {
48      String lower = sql.toLowerCase();
49      String[] blacklist = {"drop", "delete", "update", "alter", "insert",
50  "truncate"};
51      for (String word : blacklist) {

```

```
49         if (lower.contains(word)) return false;
50     }
51     if (!lower.contains("limit")) return false;
52     return lower.contains("?");
53 }
54
55     public static class SqlResp {
56         public String sql;
57         public String[] params;
58     }
59 }
```

七、模块五：双数据源融合与答案生成

7.1 数据优先级机制

结构化数据库中的精确参数数据（版本号、Star数、功能矩阵等）权威性最高，是事实标准；爬虫文档用于补充操作教程、代码示例、背景说明。

冲突处理规则：当数据库数据和文档数据不一致时，强制以数据库精确数据为准，避免文档过时信息误导。

7.2 生成约束 Prompt

增强版系统提示词

- 1 你是本地知识库问答助手，严格遵守以下规则：
- 2 1. 只能使用下方参考资料中的信息，绝对不能编造、臆测、补充参考资料中没有的内容；
- 3 2. 数据库结构化数据（标注为【数据库精确数据】）优先级最高，是事实标准；
- 4 3. 爬虫文档资料（标注为【爬虫文档参考资料】）用于补充说明、教程、示例；
- 5 4. 如果数据库数据和文档数据有冲突，以数据库精确数据为准；
- 6 5. 如果参考材料中没有答案，明确告诉用户"当前知识库暂无相关信息"，不要强行回答；
- 7 6. 回答要清晰、有条理，适当使用分点和小标题；
- 8 7. 回答末尾标注信息来源类型。

八、完整 RAG 流水线整合

8.1 完整执行流程

1. **查询增强：**LLM 生成多条优化后的检索文本
2. **并行检索：**向量多路召回 + SQL 结构化数据查询
3. **文档重排：**Rerank 过滤无关文档，保留高相关片段

4. 上下文构建：拼接结构化数据 + 文档资料，标注优先级

5. 答案生成：约束型 Prompt 送入 LLM 生成带溯源的回答

8.2 核心流水线代码

RagPipeline.java 核心流水线

```
1  package com.rag.pipeline;
2
3  import com.rag.config.RagConfig;
4  import com.rag.enhance.QueryEnhancer;
5  import com.rag.enhance.Reranker;
6  import com.rag.llm.OllamaClient;
7  import com.rag.sql.SqlGenerator;
8  import com.rag.sql.SqlExecutor;
9  import com.rag.vector.ChromaVectorStore;
10
11 import java.util.ArrayList;
12 import java.util.List;
13 import java.util.Map;
14
15 public class RagPipeline {
16     private final QueryEnhancer queryEnhancer = new QueryEnhancer();
17     private final ChromaVectorStore vectorStore = new ChromaVectorStore();
18     private final Reranker reranker = new Reranker();
19     private final SqlGenerator sqlGenerator = new SqlGenerator();
20     private final SqlExecutor sqlExecutor = new SqlExecutor();
21     private final OllamaClient ollama = new OllamaClient();
22
23     /**
24      * 执行完整的增强 RAG 查询
25      */
26     public String doRag(String userQuery) throws Exception {
27         // 1. 查询增强 - 多路召回
28         List<String> enhanceQueries = queryEnhancer.enhance(userQuery);
29         List<String> allRawDocs = new ArrayList<>();
30         for (String q : enhanceQueries) {
31             List<String> docs = vectorStore.search(q, RagConfig.VECTOR_TOP_K);
32             allRawDocs.addAll(docs);
33         }
34         // 去重
35         List<String> uniqueDocs = new ArrayList<>();
36         for (String d : allRawDocs) {
37             if (!uniqueDocs.contains(d)) uniqueDocs.add(d);
38         }
39     }
40 }
```

```

40         // 2. Rerank 重排序
41         List<String> topDocs = reranker.rerank(userQuery, uniqueDocs,
RagConfig.RERANK_KEEP_NUM);
42
43         // 3. 动态 SQL 查询结构化数据
44         List<Map<String, Object>> dbData = new ArrayList<>();
45         SqlGenerator.SqlResp sqlResp = sqlGenerator.generate(userQuery);
46         if (sqlGenerator.securityCheck(sqlResp.sql)) {
47             dbData = sqlExecutor.execute(sqlResp.sql, sqlResp.params);
48         }
49
50         // 4. 构建上下文 (DB 数据优先)
51         StringBuilder context = new StringBuilder();
52         context.append("【数据库精准结构化数据】
\n").append(dbData).append("\n\n");
53         context.append("【爬虫文档参考资料】 \n");
54         for (String d : topDocs) context.append(d).append("\n---\n");
55
56         // 5. LLM 生成最终回答
57         String sysPrompt = """
58 你是本地知识库问答助手，严格遵守：
59 1. 只能使用下方参考资料，资料不存在不能编造；
60 2. 数据库结构化数据权威性高于文档，冲突以数据库为准；
61 3. 无相关资料直接回复：暂无知识库信息。
62
63 参考资料：
64 %s
65 用户问题：%s
66 """.formatted(context, userQuery);
67
68         return ollama.chat(sysPrompt);
69     }
70 }

```

九、项目结构与依赖

9.1 完整目录结构

Java 项目结构

```

1  java-dual-rag/
2  |— pom.xml           # Maven 配置
3  |— README.md        # 项目说明
4  |— src/main/
5  |   |— java/com/rag/

```

```

6      |   | └─ Main.java           # 主入口
7      |   | └─ config/
8      |   |   └─ RagConfig.java    # 全局配置
9      |   | └─ llm/
10     |   |   └─ OllamaClient.java # Ollama 客户端封装
11     |   | └─ vector/
12     |   |   └─ ChromaVectorStore.java # 向量存储操作
13     |   | └─ sql/
14     |   |   └─ SqlGenerator.java  # SQL 动态生成
15     |   |   └─ SqlExecutor.java  # SQL 执行器
16     |   | └─ enhance/
17     |   |   └─ QueryEnhancer.java # 查询增强
18     |   |   └─ Reranker.java      # 文档重排序
19     |   | └─ pipeline/
20     |   |   └─ RagPipeline.java   # RAG 流水线（核心）
21     |   |   └─ util/
22     |   |       └─ TextUtils.java # 文本工具类
23     |   └─ resources/
24     |       └─ sample_docs.txt    # 示例文档

```

9.2 Maven 依赖

pom.xml 核心依赖

```

1  <dependencies>
2      <!-- JSON 处理 -->
3      <dependency>
4          <groupId>com.alibaba.fastjson2</groupId>
5          <artifactId>fastjson2</artifactId>
6          <version>2.0.32</version>
7      </dependency>
8
9      <!-- SQLite 数据库 -->
10     <dependency>
11         <groupId>org.xerial</groupId>
12         <artifactId>sqlite-jdbc</artifactId>
13         <version>3.46.0.1</version>
14     </dependency>
15
16     <!-- Chroma 向量数据库客户端 -->
17     <dependency>
18         <groupId>io.github.trallnurture</groupId>
19         <artifactId>chromadb-java-client</artifactId>
20         <version>0.1.10</version>
21     </dependency>
22

```

```
23      <!-- 工具类 -->
24      <dependency>
25          <groupId>org.apache.commons</groupId>
26          <artifactId>commons-lang3</artifactId>
27          <version>3.14.0</version>
28      </dependency>
29  </dependencies>
```

十、部署与运行

10.1 快速启动步骤

1. 启动 Ollama 服务：确保 qwen3 和 bge-m3 模型已拉取
2. 启动 Chroma 服务：`chroma run --path ./chroma_data`
3. 编译项目：`mvn clean package -DskipTests`
4. 初始化数据：`java -jar target/java-dual-rag.jar --init`
5. 运行查询：`java -jar target/java-dual-rag.jar --query "你的问题"`

10.2 性能优化建议

- **缓存层**：对高频查询结果做缓存，减少重复推理
- **Embedding 缓存**：相同文本的向量结果缓存，减少 Embedding 模型调用
- **并行检索**：SQL 查询和向量检索可改为并发执行，缩短响应时间
- **专业 Reranker**：替换为 bge-reranker 等专用模型，速度更快
- **向量数据库升级**：数据量大时替换为 Milvus 或 Qdrant

十一、常见问题与避坑指南

11.1 检索不准怎么办

- 检查文档切片是否合理，代码类文档建议增大 chunk 到 600+
- 增加查询增强的生成数量，从 3 条提升到 5 条
- 调整 Rerank 保留数量，适当增加 Top N
- 检查 Embedding 模型是否适配中文，推荐 bge-m3

11.2 LLM 幻觉（编造内容）

- 降低 temperature 到 0，减少模型随机性

- 加强系统 Prompt 约束，明确禁止编造
- 增加答案自检环节：生成后再调用一次 LLM 判断是否超出参考资料
- 结构化数据优先，减少纯文档回答的不确定性

11.3 SQL 生成失败

- 在 Prompt 中增加 Few-shot 示例，提供更多样例
- Schema 描述更详细，补充字段说明和业务规则
- 增加降级兜底逻辑，生成失败时使用模糊查询

11.4 响应速度慢

- 使用更小更快的模型做路由和 Rerank，大模型只做最终生成
- 增加缓存层，高频查询直接返回缓存结果
- 向量检索和 SQL 查询并行执行
- GPU 加速 Ollama 推理

十二、进阶扩展方向

- **多模态增强**：接入多模态 LLM 解析图片教程，转为文字存入向量库
- **自检索迭代**：第一轮回答后，LLM 自检缺失信息，自动生成补充检索词二次检索
- **自适应上下文裁剪**：根据模型窗口大小自动调整检索文档数量
- **用户反馈优化**：收集用户点赞/点踩，优化检索排序和切片策略
- **Spring Boot 封装**：封装为 HTTP 接口，提供 Web API 服务

✅ **总结**：增强 RAG 的核心是「召回要全、排序要准、生成要严」。四大模块层层递进，从查询端优化到生成端约束，系统性地提升知识库问答的准确率和可用性。

(注：部分内容可能由 AI 生成)